

Bắt đầu lập trình hoặc tạo mã bằng trí tuệ nhân tạo (AI).

✎ Bài giảng về định thức qua Python,

Prof Happy AI dịch (2026)

```
import numpy as np
import sympy as sy
sy.init_printing()
import matplotlib.pyplot as plt
plt.style.use('ggplot')
```

```
np.set_printoptions(precision=3)
np.set_printoptions(suppress=True)
```

```
def round_expr(expr, num_digits):
    return expr.xreplace({n : round(n, num_digits) for n in expr.atoms(sy.Number)})
```

✎ Trục quan hóa Định thức

Ý nghĩa vật lý của định thức là tính diện tích được bao bởi các vectơ. Ví dụ, xét ma trận:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Định thức đại diện cho diện tích của hình bình hành được tạo bởi các vectơ:

$$\begin{bmatrix} a \\ c \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} b \\ d \end{bmatrix}$$

Ở đây chúng ta minh họa với một ma trận:

$$\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

Cũng rất dễ hiểu khi diện tích được tạo bởi hai vectơ này thực chất là một hình chữ nhật.

```
matrix = np.array([[2, 0], [0, 3]])
def plot_2ddet(matrix):
    # Calculate the determinant of the matrix
    det = np.linalg.det(matrix)

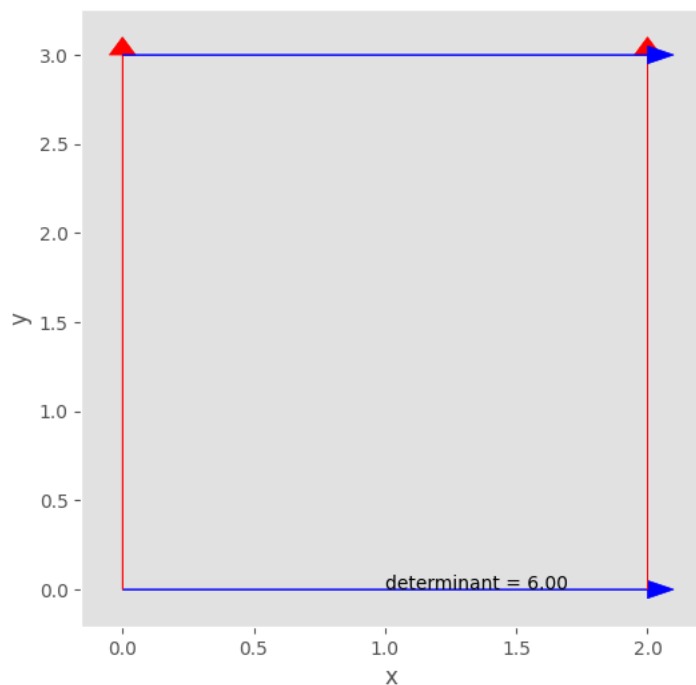
    # Create a figure and axis
    fig, ax = plt.subplots(figsize=(6, 6))

    # Plot the vectors
    ax.arrow(0, 0, matrix[0,0], matrix[0,1], head_width=0.1, head_length=0.1, color='b')
    ax.arrow(0, 0, matrix[1,0], matrix[1,1], head_width=0.1, head_length=0.1, color='r')
    ax.arrow(matrix[0,0], matrix[0,1], matrix[1,0], matrix[1,1], head_width=0.1, head_length=0.1, color='r')
    ax.arrow(matrix[1,0], matrix[1,1], matrix[0,0], matrix[0,1], head_width=0.1, head_length=0.1, color='b')

    # Annotate the determinant value
    ax.annotate(f'determinant = {det:.2f}', (matrix[0,0]/2, matrix[0,1]/2))

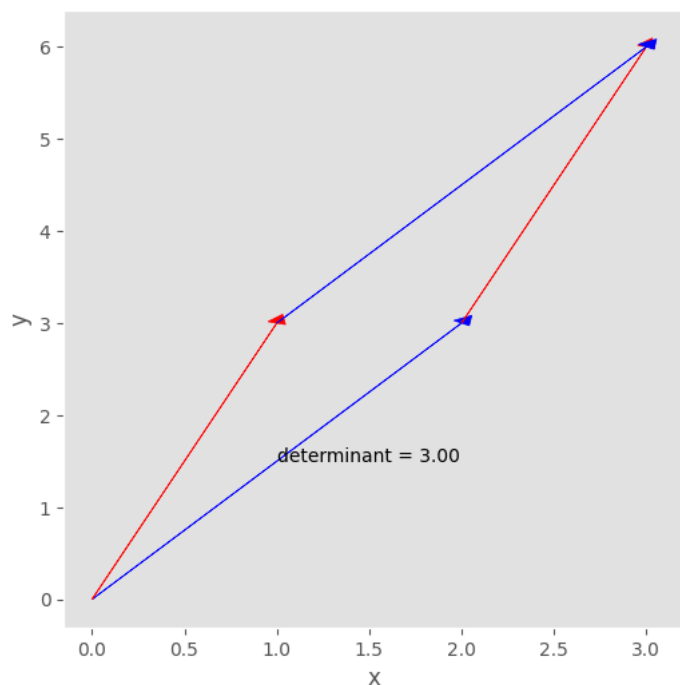
    # Add labels and show the plot
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.grid()
    plt.show()

if __name__ == '__main__':
    plot_2ddet(matrix)
```



Nếu ma trận không phải là ma trận đường chéo, diện tích sẽ có dạng một hình bình hành. Tương tự, trong không gian 3 chiều, định thức là thể tích của một hình khối song song (parallelepiped). Đầu tiên, hãy vẽ một hình bình hành.

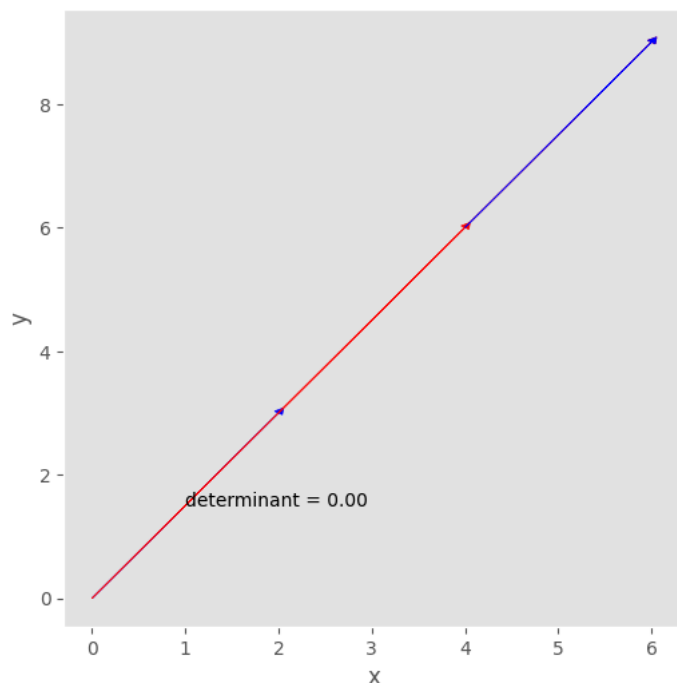
```
matrix = np.array([[2, 3], [1, 3]])
plot_2ddet(matrix)
```



Tuy nhiên, định thức có thể là một số âm, điều đó có nghĩa là chúng ta lật ngược diện tích giống như lật một tờ giấy.

Điều gì xảy ra nếu hai vectơ phụ thuộc tuyến tính? Diện tích giữa các vectơ sẽ bằng không.

```
matrix = np.array([[2, 3], [4, 6]])
plot_2ddet(matrix)
```



Đây chính là lý do tại sao nếu chúng ta muốn một ma trận có hạng đầy đủ (full rank) hoặc các cột độc lập tuyến tính, định thức không được bằng không!

Dưới đây là biểu đồ của một hình khối song song, trong trường hợp bạn không chắc nó trông như thế nào.

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')

# Define the vertices of the parallelepiped
vertices = np.array([[2, 0, 0], [2, 3, 0], [0, 3, 0], [0, 0, 0],
                    [2, 3, 4], [2, 6, 4], [0, 6, 4], [0, 3, 4]])

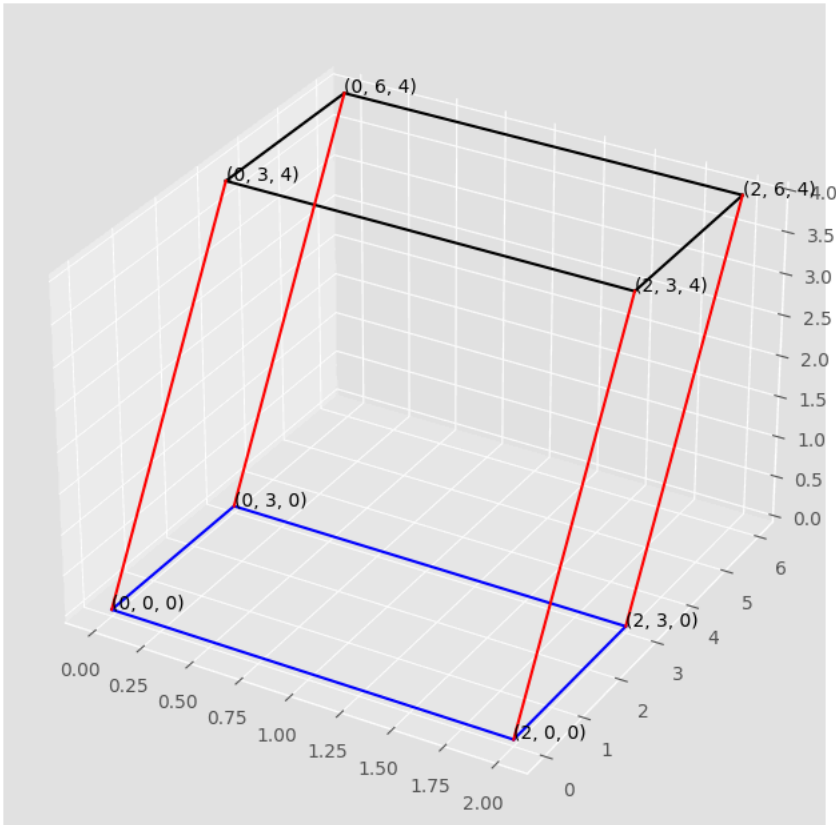
# Plot edges between corresponding vertices
def plot_edges(vertices, pairs, color):
    for (i, j) in pairs:
        ax.plot([vertices[i, 0], vertices[j, 0]],
                [vertices[i, 1], vertices[j, 1]],
                [vertices[i, 2], vertices[j, 2]], color=color)

# Pairs of indices representing edges to be plotted
bottom_edges = [(i, (i+1)%4) for i in range(4)]
top_edges = [(i+4, (i+1)%4+4) for i in range(4)]
vertical_edges = [(i, i+4) for i in range(4)]

# Plot all edges
plot_edges(vertices, bottom_edges, 'blue')
plot_edges(vertices, top_edges, 'k')
plot_edges(vertices, vertical_edges, 'red')

# Add annotations for the coordinates at each vertex
for vertex in vertices:
    ax.text(vertex[0], vertex[1], vertex[2], f"({vertex[0]}, {vertex[1]}, {vertex[2]})")

plt.show()
```



▼ Tính định thức

Cho ma trận $A 2 \times 2$, giải thuật tính định thức là

$$A = \begin{vmatrix} a & b \\ c & d \end{vmatrix} \quad \text{tương đương với} \quad \det A = ad - bc$$

Bây giờ chúng ta thử nghiệm với SymPy

```
a, b, c, d, e, f, g, h, i = sy.symbols('a, b, c, d, e, f, g, h, i', real = True)
```

Với các ký hiệu đã định nghĩa, các giải thuật của định thức 2×2 và 3×3 là:

```
A = sy.Matrix([[a, b], [c, d]])
A.det()
```

Bất kỳ thứ gì khác 2×2 đều phức tạp

```
B = sy.Matrix([[a, b, c], [d, e, f], [g, h, i]])
B.det()
```

```
aei - afh - bdi + bfg + cdh - ceg
```

▼ Khai triển Cofactor

Cofactor (i, j) - của A ký hiệu là C_{ij} và được cho bằng công thức

$$C_{ij} = (-1)^{i+j} \det A_{ij} = (-1)^{i+j} M_{ij}$$

với M_{ij} là định thức con (minor) có được bằng cách loại bỏ hàng thứ i và cột thứ j .

Hãy học từ ví dụ. Xét ma trận A :

$$A = \begin{bmatrix} 1 & 5 & 0 \\ 2 & 4 & -1 \\ 0 & -2 & 0 \end{bmatrix}$$

Bất kỳ định thức nào cũng có thể được khai triển dọc theo một hàng hoặc một cột bất kỳ.

$$\begin{aligned} \det A &= 1 \cdot \det \begin{bmatrix} 4 & -1 \\ -2 & 0 \end{bmatrix} - 5 \cdot \det \begin{bmatrix} 2 & -1 \\ 0 & 0 \end{bmatrix} + 0 \cdot \det \begin{bmatrix} 2 & 4 \\ 0 & -2 \end{bmatrix} \\ &= 1 \cdot (0 - (-2)) - 5 \cdot (0 - 0) + 0 \cdot (-4 - 0) \\ &= 1 \cdot 2 - 5 \cdot 0 + 0 \cdot (-4) \\ &= 2 - 0 + 0 \\ &= 2 \end{aligned}$$

Các hệ số 1, 5 và 0 đứng trước mỗi định thức con là các phần tử của hàng đầu tiên của A .

Nói chung, khai triển qua dòng i -th hay cột j -th là:

$$\begin{aligned} \det A &= a_{i1}C_{i1} + a_{i2}C_{i2} + \cdots + a_{in}C_{in} \\ \det A &= a_{1j}C_{1j} + a_{2j}C_{2j} + \cdots + a_{nj}C_{nj} \end{aligned}$$

▼ Ví dụ dùng SymPy để khai triển định thức

Khảo sát ma trận sau để thực hiện khai triển cofactor

```
A = sy.Matrix([[49, 0, 61], [73, 22, 96], [2, 0, 32]]); A
```

$$\begin{bmatrix} 49 & 0 & 61 \\ 73 & 22 & 96 \\ 2 & 0 & 32 \end{bmatrix}$$

Khai triển cofactor với cột có chứa hai số không sẽ giúp giảm bớt gánh nặng tính toán nhất.

$$\det A = a_{12}(-1)^{1+2}C_{12} + a_{22}(-1)^{2+2}C_{22} + a_{32}(-1)^{3+2}C_{32}$$

Chúng ta có thể dùng hàm SymPy để tính các định thức con (minors): `sy.matrices.matrices.MatrixDeterminant.minor(A, i, 1)`.

Chúng ta cũng định nghĩa hàm để khai triển cofactor:

```
def cof_exp(matrix, c):
    """Calculate the cofactor expansion of a matrix along the c-th column."""
    detA = 0
    for i in range(matrix.shape[0]): # matrix.shape[0] is the total number of rows
        minor_matrix = matrix.minor_submatrix(i, c)
        cofactor = (-1)**(i + c) * minor_matrix.det()
        detA += matrix[i, c] * cofactor
    return detA
```

```
cof_exp(A,1)
```

Có thể dễ dàng xác minh thuật toán khai triển bằng hàm tính định thức của SymPy..

```
A.det()
```

```
31812
```

Thực tế, bạn có thể thử nghiệm với bất kỳ ma trận ngẫu nhiên nào có nhiều số không; hàm số bên dưới có tham số percent=70, nghĩa là 70% các phần tử trong ma trận là các số khác không.

```
B = sy.randMatrix(r = 7, min=10, max=50, percent=70);B
```

$$\begin{bmatrix} 12 & 30 & 0 & 40 & 0 & 28 & 0 \\ 0 & 19 & 0 & 36 & 0 & 19 & 0 \\ 16 & 21 & 40 & 15 & 0 & 50 & 20 \\ 16 & 21 & 48 & 0 & 44 & 0 & 0 \\ 10 & 10 & 21 & 25 & 0 & 39 & 0 \\ 36 & 11 & 14 & 32 & 49 & 21 & 0 \\ 37 & 50 & 44 & 11 & 21 & 18 & 0 \end{bmatrix}$$

Tính định thức với hàm tự định nghĩa

```
cof_exp(B, 1)
```

```
-6635900400
```

Sau đó, hãy xác minh kết quả bằng cách sử dụng phương thức tính định thức ``.det()```. Chúng ta có thể thấy khai triển cofactor làm việc!

```
B.det()
```

```
-6635900400
```

Các ma trận con được trích bằng mã lệnh `.minor_submatrix()`, ví dụ, ma trận M_{23} của B là

```
B.minor_submatrix(1, 2)
```

$$\begin{bmatrix} 12 & 30 & 40 & 0 & 28 & 0 \\ 16 & 21 & 15 & 0 & 50 & 20 \\ 16 & 21 & 0 & 44 & 0 & 0 \\ 10 & 10 & 25 & 0 & 39 & 0 \\ 36 & 11 & 32 & 49 & 21 & 0 \\ 37 & 50 & 11 & 21 & 18 & 0 \end{bmatrix}$$

Ma trận *matrix* là ma trận chứa tất cả cofactors của ma trận gốc và hàm `.cofactor_matrix()` để dàng sinh các loại ma trận.

$$A = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix} = \begin{bmatrix} (-1)^{1+1}M_{11} & (-1)^{1+2}M_{12} & (-1)^{1+3}M_{13} \\ (-1)^{2+1}M_{21} & (-1)^{2+2}M_{22} & (-1)^{2+3}M_{23} \\ (-1)^{3+1}M_{31} & (-1)^{3+2}M_{32} & (-1)^{3+3}M_{33} \end{bmatrix}$$

```
A.cofactor_matrix()
```

$$\begin{bmatrix} 704 & -2144 & -44 \\ 0 & 1446 & 0 \\ -1342 & -251 & 1078 \end{bmatrix}$$

▼ Ma trận tam giác

Nếu A là ma trận tam giác, việc khai triển cofactor có thể áp dụng theo cách lặp, kết quả sẽ là tích của các phần tử trên đường chéo chính.

$$\det A_{n \times n} = \prod_{i=1}^n a_{ii}$$

với a_{ii} là phần tử trên đường chéo.

Đây là chứng minh, bắt đầu bằng A

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \cdots & a_{2n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{bmatrix}$$

Khai triển Cofactor trên cột thứ nhất, dấu của a_{ii} luôn dương vì $(-1)^{2i}$

$$a_{11} \cdot \begin{bmatrix} a_{22} & a_{22} & \cdots & a_{2n} \\ & a_{33} & \cdots & a_{3n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{bmatrix}$$

Tiếp tục khai triển cofactor

$$\det A = a_{11} a_{22} \cdot \begin{bmatrix} a_{33} & a_{34} & \cdots & a_{3n} \\ & a_{44} & \cdots & a_{4n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{bmatrix}$$

Iterating the expansion, eventually

$$\det A = a_{11} \cdots a_{n-2,n-2} \cdot \begin{bmatrix} a_{n-1,n-1} & a_{n-1,n} \\ & a_{nn} \end{bmatrix} = a_{11} \cdots a_{nn}$$

Chúng ta kiểm chứng qua ví dụ số, tạo ma trận tam gian trên bằng số ngẫu nhiên.

```
A = np.round(np.random.rand(5,5)*100)
A_triu = np.triu(A); A_triu
```

```
array([[79., 66., 59., 58., 33.],
       [ 0., 88., 17., 39., 72.],
       [ 0.,  0., 27., 86., 57.],
       [ 0.,  0.,  0., 47., 73.],
       [ 0.,  0.,  0.,  0., 58.]])
```

Tiếp tục với định thức `np.linalg.det`

```
np.linalg.det(A_triu)
```

Trích đường chéo với hàm `np.diag()`, rồi tính tích. Chúng ta mang có cùng kết quả.

```
A_diag = np.diag(A_triu)
np.prod(A_diag)
```

▼ Các thuộc tính của Định thức

Định thức có một danh sách dài các thuộc tính, nhưng chúng chủ yếu được dẫn xuất từ khai triển cofactor. Không cần thiết phải học thuộc lòng chúng.

1. Let A be an $n \times n$ square matrix. If one row of A is multiplied by k to produce the matrix B , then $\det B = k \det A$.
2. Let A be an $n \times n$ square matrix. If two rows of A are interchanged to produce a matrix B , then $\det B = -\det A$.
3. Let A be an $n \times n$ square matrix. If a multiple of one row of A is added to another row to produce the matrix B , then $\det A = \det B$.
4. If A is an $n \times n$ matrix, then $\det A^T = \det A$.
5. A square matrix A is invertible if and only if $\det A \neq 0$.
6. If A and B are $n \times n$ matrices, then $\det AB = (\det A)(\det B)$.
7. If A is an $n \times n$ matrix and k is a scalar, then $\det kA = k^n \det A$.
8. If A is an invertible square matrix, then $\det A^{-1} = \frac{1}{\det A}$.

Tất cả các thuộc tính này đều rất dễ hiểu. Chìa khóa là chứng minh chúng bằng cách sử dụng khai triển cofactor. Sau đây là vài chứng minh

Proof of property 6:

$$\begin{aligned} |AB| &= |E_p \cdots E_1 B| = |E_p| |E_{p-1} \cdots E_1 B| = \cdots \\ &= |E_p| \cdots |E_1| |B| = \cdots = |E_p \cdots E_1| |B| \\ &= |A| |B| \end{aligned}$$

Proof of property 7:

Because $\det B = k \det A$, one row of A is multiplied by k to produce B . Then multiply all the rows of A by k , there will be n k 's in front of $\det A$, which is $k^n \det A$.

Proof of property 8:

$$\begin{aligned} AA^{-1} &= I \\ |AA^{-1}| &= |I| \\ |A| |A^{-1}| &= 1 \\ |A^{-1}| &= \frac{1}{|A|} \end{aligned}$$

These properties are useful in the analytical derivation of other theorems; however, they are not efficient for numerical computation.

▼ Quy tắc Cramer

If a linear system has n equations and n variables, an algorithm called *Cramer's Rule* can solve the system in terms of determinants as long as the solution is unique.

$$A_{n \times n} \mathbf{b}_n = \mathbf{x}_n$$

Some convenient notations are introduced here:

For any $A_{n \times n}$ and vector $\mathbf{b}$, denote $A_i(\mathbf{b})$ as the matrix obtained from replacing the i th column of A by $\mathbf{b}$.

$$A_i(\mathbf{b}) = [\mathbf{a}_1 \quad \cdots \quad \mathbf{b} \quad \cdots \quad \mathbf{a}_n]$$

The Cramer's Rule can solve each x_i without solving the whole system

$$x_i = \frac{\det A_i(\mathbf{b})}{\det A}, \quad i = 1, 2, \dots, n$$

Fast Proof of Cramer's Rule:

$$\begin{aligned} A \cdot I_i(\mathbf{x}) &= A [\mathbf{e}_1 \quad \cdots \quad \mathbf{x} \quad \cdots \quad \mathbf{e}_n] = [A\mathbf{e}_1 \quad \cdots \quad A\mathbf{x} \quad \cdots \quad A\mathbf{e}_n] \\ &= [\mathbf{a}_1 \quad \cdots \quad \mathbf{b} \quad \cdots \quad \mathbf{a}_n] = A_i(\mathbf{b}) \end{aligned}$$

where $I_i(\mathbf{x})$ is an identity matrix whose i -th column replaced by $\mathbf{x}$. With determinant's property,

$$(\det A) (\det I_i(\mathbf{x})) = \det A_i(\mathbf{b})$$

$\det I_i(\mathbf{x}) = x_i$, can be shown by cofactor expansion.

▼ Ví dụ dùng cho quy tắc Cramer

Khảo sát hệ phương trình:

$$\begin{aligned} 2x - y + 3z &= -3 \\ 3x + 3y - z &= 10 \\ -x - y + z &= -4 \end{aligned}$$

You have surely known several ways to solve it, but let's test if Cramer's rule works.

Input the matrices into NumPy arrays.

```
A = np.array([[2, -1, 3], [3, 3, -1], [-1, -1, 1]])
b = np.array([-3, 10, -4])
```



```
A_1b = np.copy(A) # Python variable is a reference tag
A_1b[:,0]=b

A_2b = np.copy(A)
A_2b[:,1]=b

A_3b = np.copy(A)
A_3b[:,2]=b
```

ặ

Theo các quy tắc Cramer:

```
x1 = np.linalg.det(A_1b)/np.linalg.det(A)
x2 = np.linalg.det(A_2b)/np.linalg.det(A)
x3 = np.linalg.det(A_3b)/np.linalg.det(A)
(x1, x2, x3)
```

```
(1.0, 2.0, -1.0)
```

Chúng ta có thể kiểm chứng kết quả bằng hàm thư viện NumPy `np.linalg.solve`.

```
np.linalg.solve(A, b)
```

```
array([ 1.,  2., -1.])
```

Bắt đầu lập trình hoặc tạo mã bằng trí tuệ nhân tạo (AI).

Hay các trực tiếp $A^{-1}b$

```
np.linalg.inv(A)@b
```

```
array([ 1.,  2., -1.])
```

Tất cả kết quả đều giống nhau!

Tuy nhiên, hãy nhớ rằng quy tắc Cramer hiếm khi được sử dụng trong thực tế để giải các hệ phương trình, vì chi phí tính toán của nó (được đo bằng số lượng các phép tính dấu phẩy động, hay flops) cao hơn nhiều so với phương pháp loại bỏ Gauss-Jordan.

▼ Công thức tính định thức cho ma trận nghịch đảo

An alternative algorithm for A^{-1} is

$$A^{-1} = \frac{1}{\det A} \begin{bmatrix} C_{11} & C_{21} & \cdots & C_{n1} \\ C_{12} & C_{22} & \cdots & C_{n2} \\ \vdots & \vdots & & \vdots \\ C_{1n} & C_{2n} & \cdots & C_{nn} \end{bmatrix}$$

where the matrix of cofactors on RHS is the *adjugate* matrix, SymPy function is `sy.matrices.matrices.MatrixDeterminant.adjugate`.

And this is the transpose of the *cofactor matrix* which we computed using

```
sy.matrices.matrices.MatrixDeterminant.cofactor_matrix
```

▼ A SymPy Example

Generate a random matrix with 20% of zero elements.

```
A = sy.randMatrix(5, min=-5, max = 5, percent = 80); A
```

Compute the adjugate matrix

$$\begin{bmatrix} 0 & -5 & 2 & 2 & -4 \\ 1 & 4 & 0 & -5 & 4 \\ 2 & 0 & 1 & 2 & 0 \\ -5 & -1 & 2 & 3 & 3 \\ 0 & 1 & 0 & 0 & 5 \end{bmatrix}$$

```
A_adjugate = A.adjugate(); A_adjugate
```

$$\begin{bmatrix} -206 & -248 & 390 & 11 & -27 \\ -73 & 14 & 78 & 34 & 90 \\ -16 & -10 & 285 & 112 & 72 \\ -214 & -253 & 294 & 67 & 9 \\ -143 & -149 & 192 & 47 & -72 \end{bmatrix}$$

We can verify if this really the adjugate of A , we pick element of $(1, 3)$, $(2, 4)$, $(5, 4)$ of A to compute the cofactors

```
display(f"(2, 0) minor: {((-1)**(1+3) * A.minor(2, 0))}") # (1, 3) cofactor of A, transposed index
display(f"(3, 1) minor: {((-1)**(2+4) * A.minor(3, 1))}")
display(f"(3, 4) minor: {((-1)**(5+4) * A.minor(3, 4))}")
```

```
'(2, 0) minor: 390'
'(3, 1) minor: 34'
'(3, 4) minor: 47'
```

Adjugate is the transpose of cofactor matrix, thus we reverse the row and column index when referring to the elements. As we have shown it is truel adjugate matrix.

Next we will check if this inverse formula works with SymPy's function.

The `sy.N()` is for converting to float approximation, i.e. if you don't like fractions.

```
A_det = A.det()
A_inv = (1/A_det)*A_adjugate
round_expr(sy.N(A_inv), 4)
```

$$\begin{bmatrix} -0.4319 & -0.5199 & 0.8176 & 0.0231 & -0.0566 \\ -0.153 & 0.0294 & 0.1635 & 0.0713 & 0.1887 \\ -0.0335 & -0.021 & 0.5975 & 0.2348 & 0.1509 \\ -0.4486 & -0.5304 & 0.6164 & 0.1405 & 0.0189 \\ -0.2998 & -0.3124 & 0.4025 & 0.0985 & -0.1509 \end{bmatrix}$$

Now again, we can verify the results with `.inv()`

```
round_expr(sy.N(A.inv()), 4)
```

$$\begin{bmatrix} -0.4319 & -0.5199 & 0.8176 & 0.0231 & -0.0566 \\ -0.153 & 0.0294 & 0.1635 & 0.0713 & 0.1887 \\ -0.0335 & -0.021 & 0.5975 & 0.2348 & 0.1509 \\ -0.4486 & -0.5304 & 0.6164 & 0.1405 & 0.0189 \\ -0.2998 & -0.3124 & 0.4025 & 0.0985 & -0.1509 \end{bmatrix}$$

Or We can show by difference.

```
A_inv-A.inv()
```

So Cramer's rule indeed works perfectly.